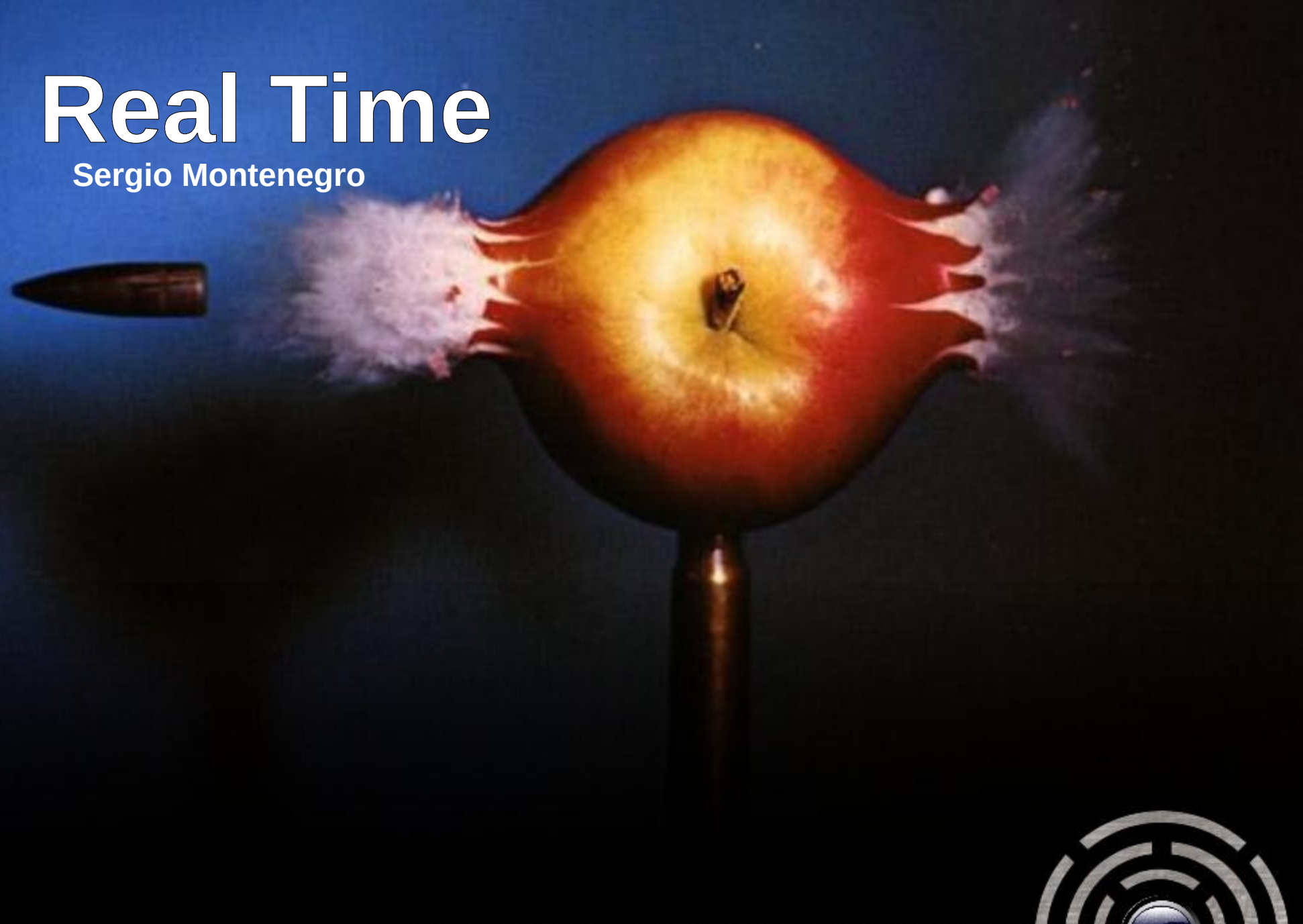
Real Time

Sergio Montenegro



Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Informationstechnik für Luft- und Raumfahrt
Aerospace Information Technology

Real Time = Fast?

Does it has to be fast?

The faster the better?

Real Time = Fast?

Does it has to be fast?

The faster the better?

no... It has to be the right time

too late is bad

too early is bad

we have

latest time

earliest time

Yes and no... Fast until the next red traffic ligth

RT or not RT?

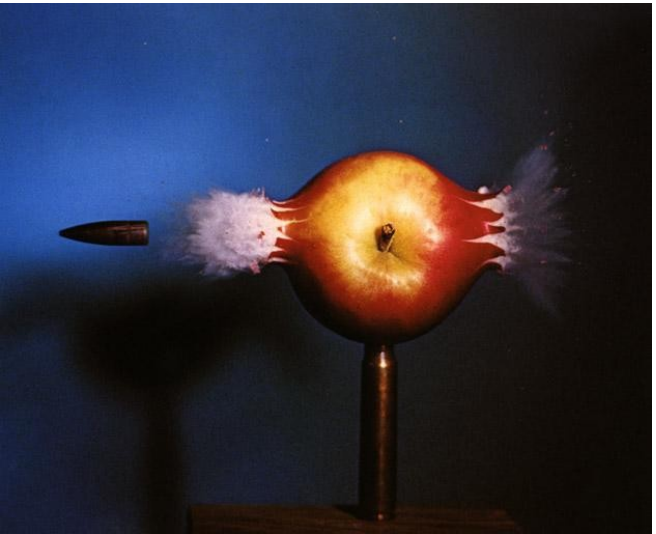
Real-time: get Ready (be fast) and **wait** for the right time
→ Much waiting, bad CPU utilization

Normal OS: Maximum CPU usage
do not wait but execute immediately

Realtime: The rigth time point

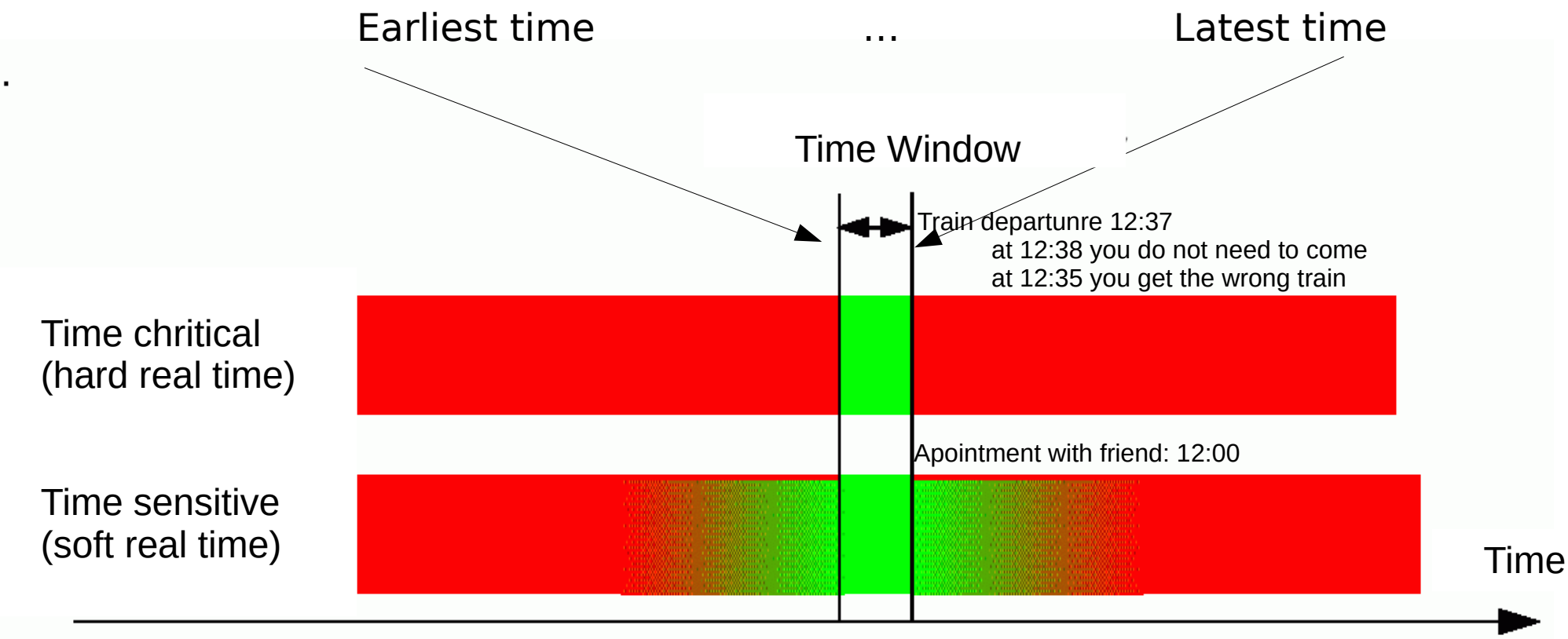
real time: not to late, but not to early

Expected results



Timings:
Some lives depend on it

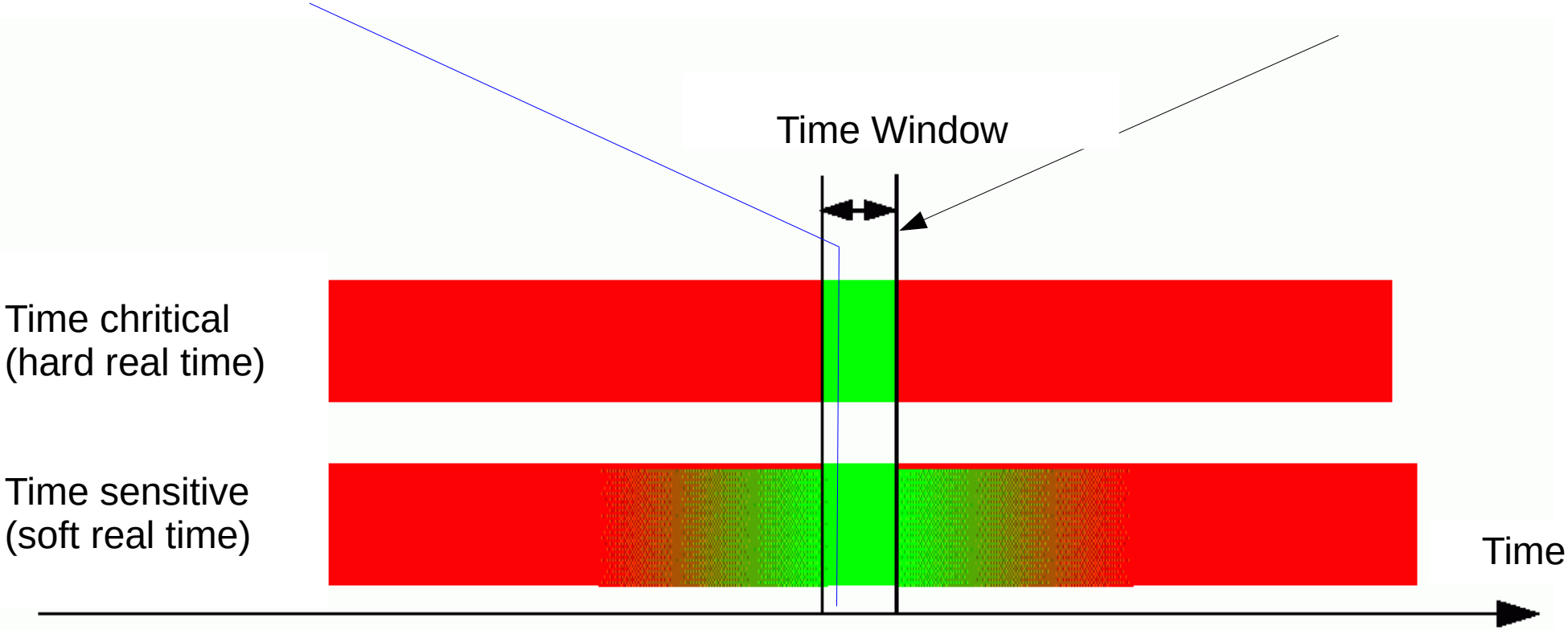
Time window



Zeitkritische und zeitempfindliche Tasks

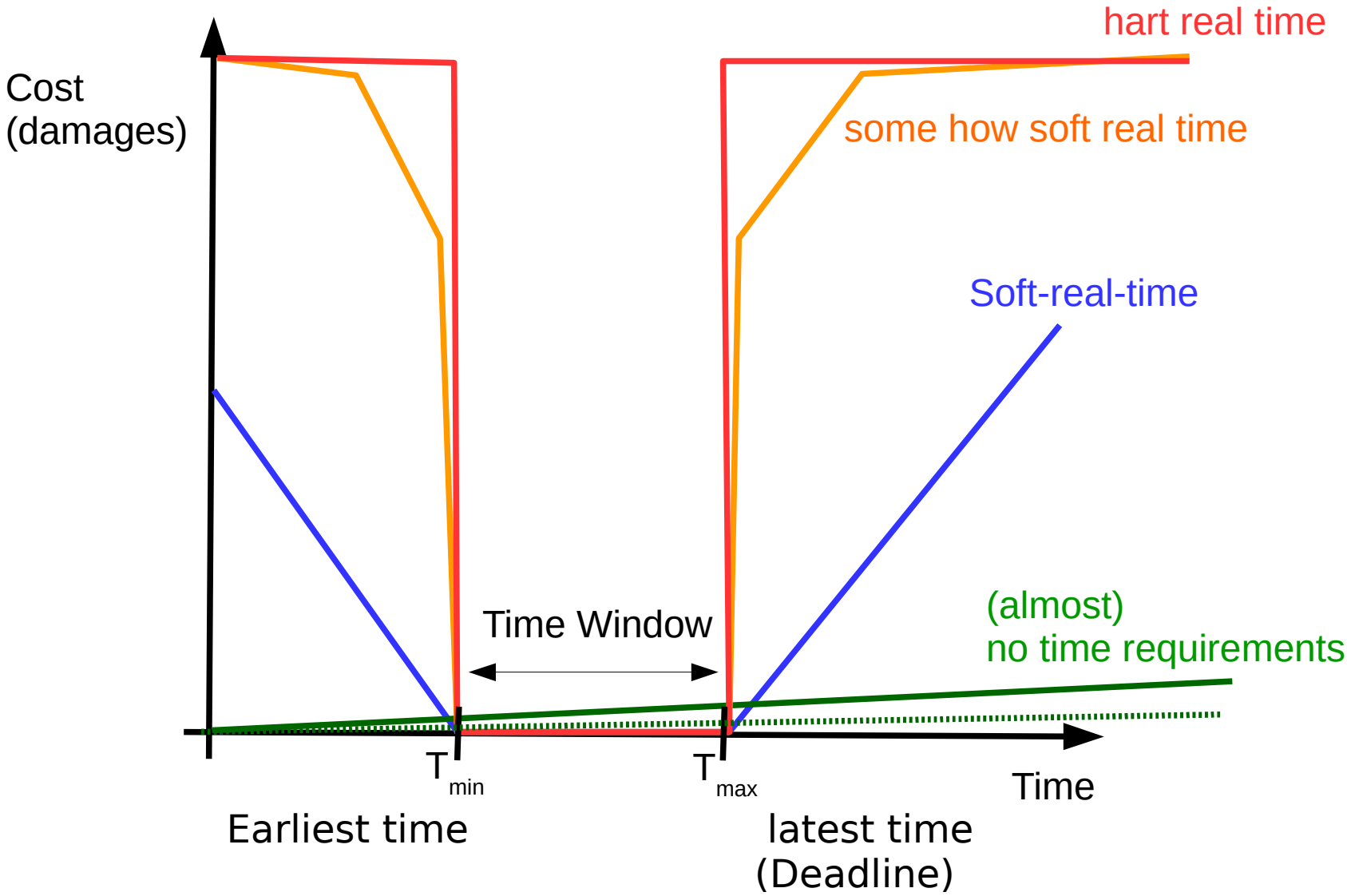
Time window

This is our target
Delay is more probable than to early



Zeitkritische und zeitempfindliche Tasks

Time window and cost



Hart / Soft Real Time

Example at the university: „exercise is due at Friday at 24:00“

Hart: After that, it will not be accepted any more

Soft: After that, the grade will be reduces one point per hour

none: please come to the exam when you thing you are ready

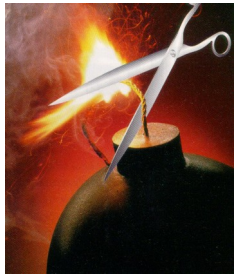
Hart: Ignition by Oto-Motor

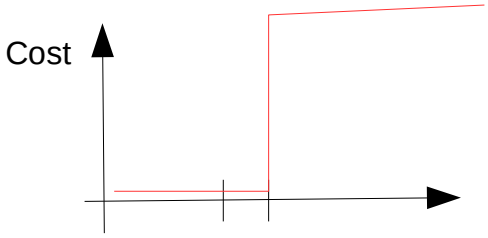
Soft: Attitude Control in satellites

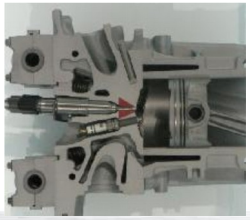
None: Text editor (using Windows)

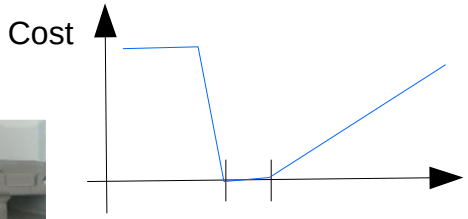
WARNING:

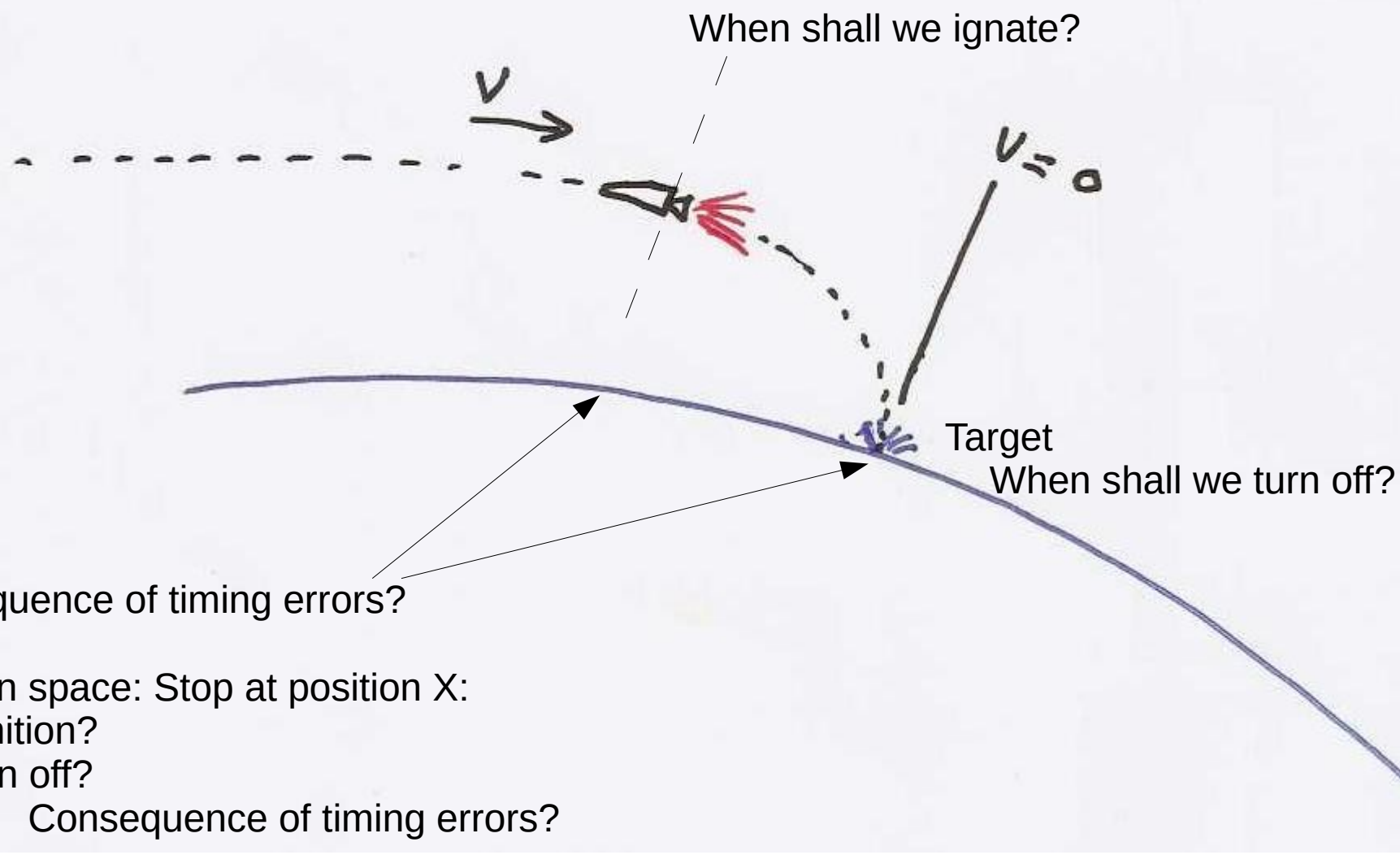
The cost function could be asymetric











Consequence of timing errors?

And... in space: Stop at position X:

- Ignition?
- turn off?
  - Consequence of timing errors?

What is real time (RT)

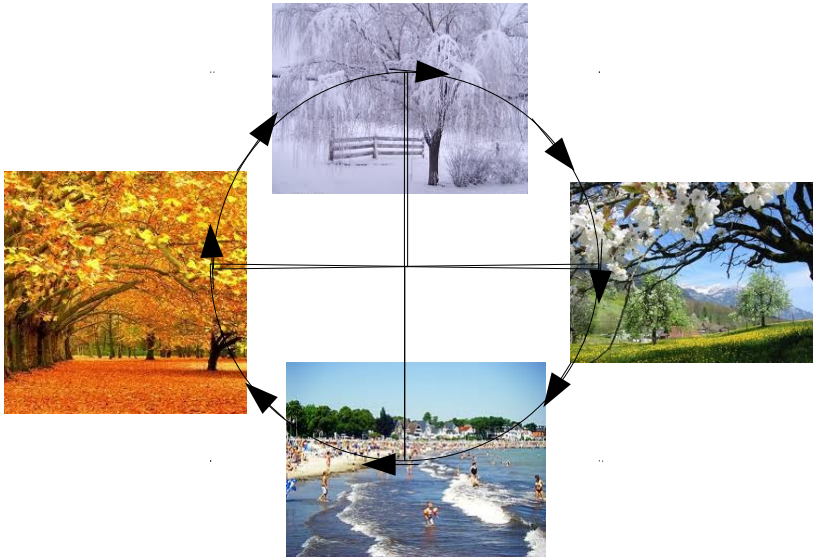
What is time?

Different times?

Linear Time
Each time point comes only once, never agian

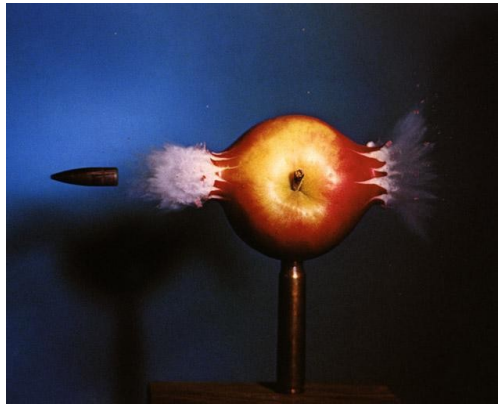
0 $1x10^6$ $2x10^6$

Cyclic time

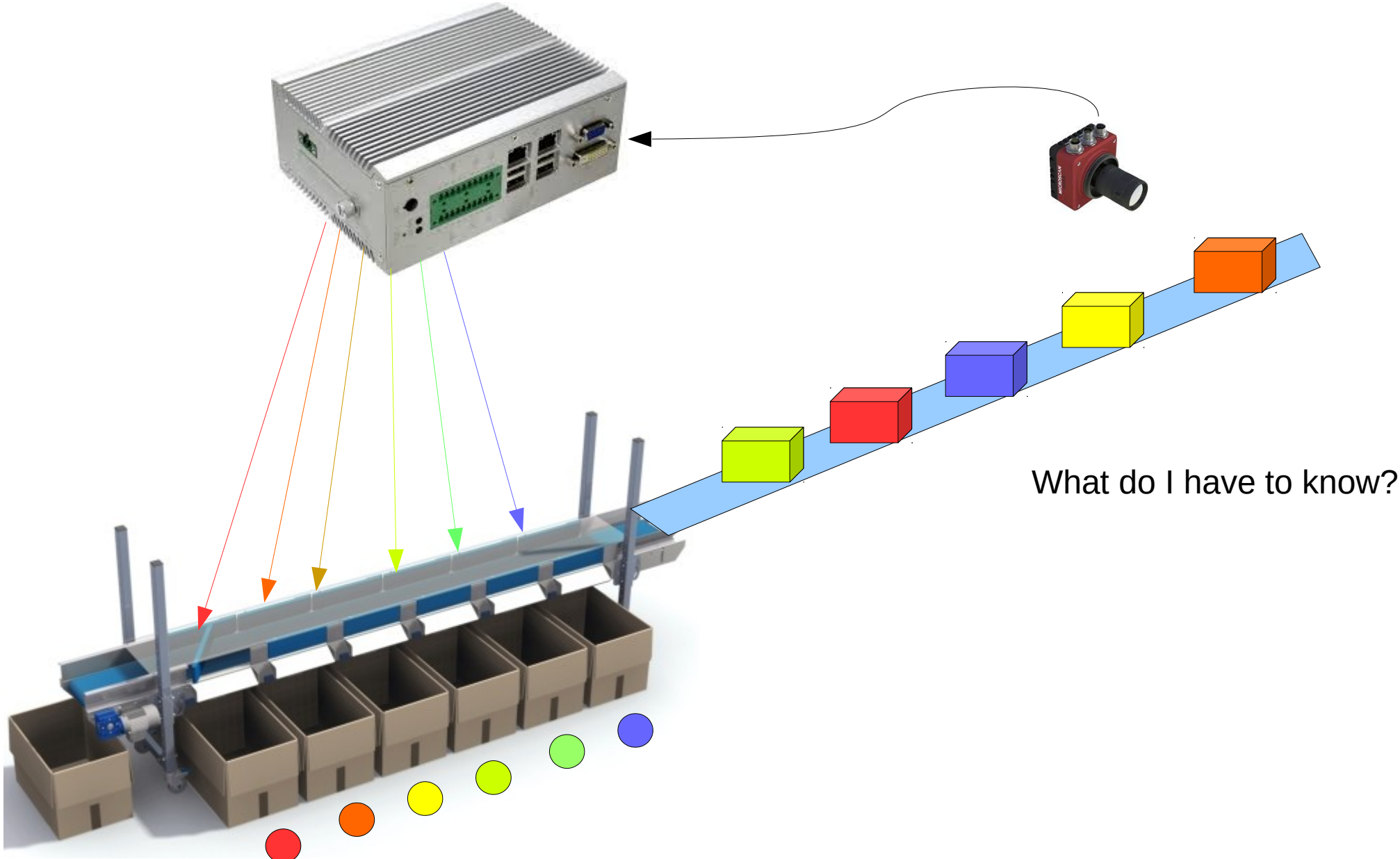


Timers
Count Down (1x)

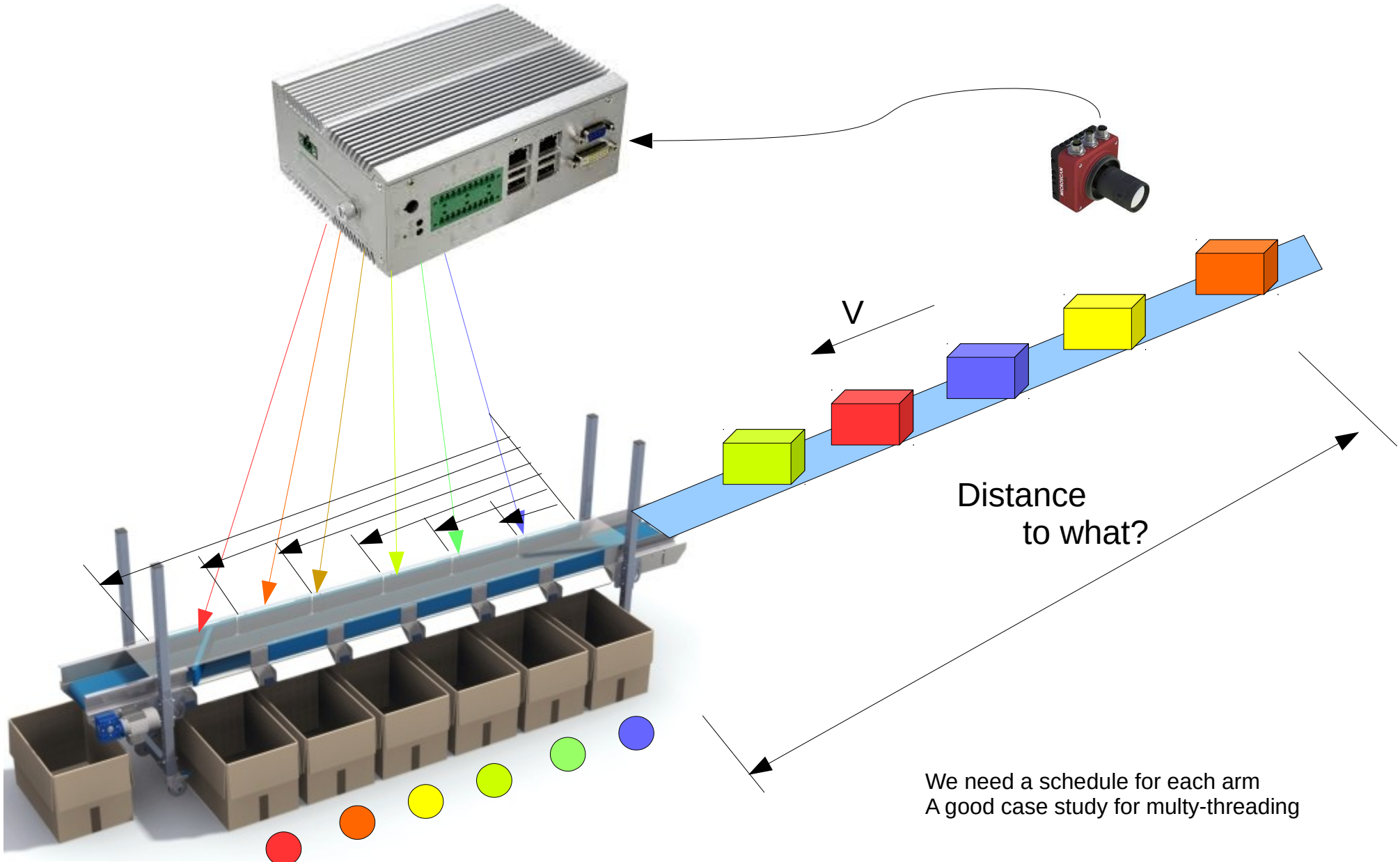
The right time
Relative to what?
Not counting, just a signal / event



How do we know when is the right time?



How do we know when is the right time?



We need a schedule for each arm
A good case study for multy-threading

The Right time

Alternatives:

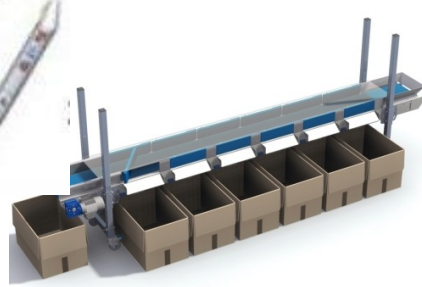
Raction

1. We have a sensor which says „NOW“ and then as soon as possible
   Reactive

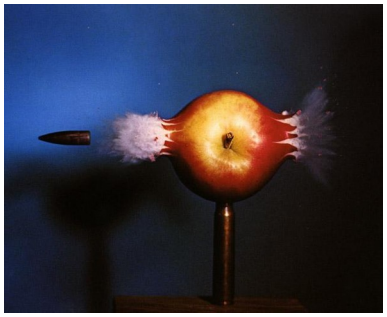
Wait

2. Precomputed: We know when to activate waht

3. Our sensor and/or computing time and/or actuator are to slow
   (or the process is really fast)
   We know the speed and the time delay from sensing to
   required reaciton: -> sensing, wait time, react



4. We have no sensor to detect the event, but we trigger the event
   -> we know speed and time daly, then like 3



5.

The Right time

Alternatives:

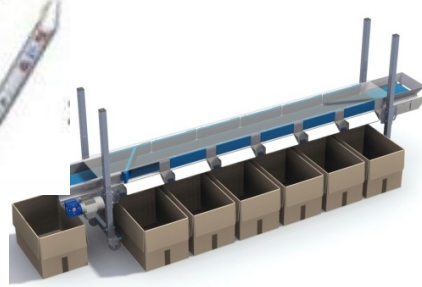
Raction

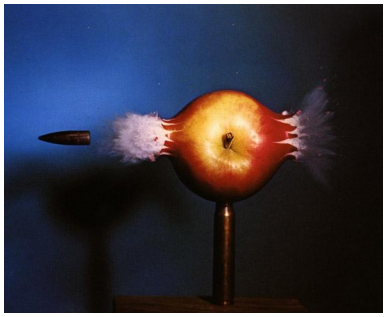
1. We have a sensor which says „NOW“ and then as soon as possible
   Reactive

Wait

2. Precomputed: We know when to activate waht

3. Our sensor and/or computing time and/or actuator are to slow
   (or the process is really fast)
   We know the speed and the time delay from sensing to
   required reaciton: -> sensing, wait time, react





4. We have no sensor to detect the event, but we trigger the event
   -> we know speed and time daly, then like 3

5. All other alternatives from 1 to 4, but periodic (cyclic)
   Question: Where is the begin of the period?
   constant period?
   Time or event control?

The Right time

How fast shall it be?
    as fast as possible?

Examples:
    How long is the period, How long (time) do we have to react?

The Right time

How fast shall it be?

as fast as possible? ... Not to late, but not to early!

Examples:

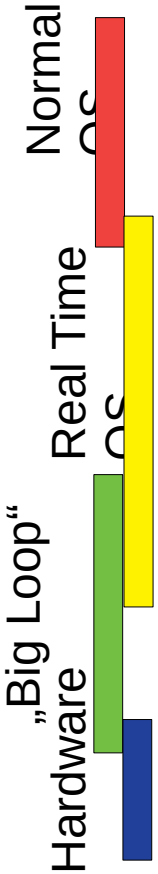
- Bank aaplications (Period 30 days, + due date)
- Alarm clock (Period: 24 Hours)
- Trafic ligth (Minuts .. Seconts)
- Satellite control attitude control (500 ms)
- Aeroplane attitude control (100 ms)
- Auto-Motor-Control (10 ms)
- Quadrocopter (3 .. 4 ms)
- Roboter-Servos, Launchers Shubvector (1 ms)
- Misiles (100 Mikrosekunden)
- Cyclotron (Mikrosekunden .. Nanosekunden)

The Right time

How fast shall it be?

as fast as possible? ... Not to late, but not to early!

Examples:



- Bank aaplications (Period 30 days, + due date)
- Alarm clock (Period: 24 Hours)
- Trafic ligth (Minuts .. Seconts)
- Satellite control attitude control (500 ms)
- Aeroplane attitude control (100 ms)
- Auto-Motor-Control (10 ms)
- Quadrocopter (4 ms)
- Roboter-Servos, Launchers Shubvector (1 ms)
- Misiles (100 Mikrosekunden)
- Cyclotron (Mikrosekunden .. Nanosekunden)

How exact can be the timing?

Jitter + **Delay**

1. Hardware	<< 1 Mikroseconds
2. Interrupt-Server Reaction to Interrupts, Including timer interrupts	3 .. 10 Mikroseconds
3. Big loop	< 1 ms
4. Time Sharing RT-OS (RT Scheduling)	3 .. 10 ms
5. Time Sharing Normal OS	> 100 ms
6. Windows + multitasking	Seconds ...

Time measurement

10^9 ns -> sec
60 sek -> min
60 min -> std
24 std -> tag
28,29,30,31 xTag -> monat
12 Monate -> jahr
365, 366 x Tag -> jahr

Counting years:
-2, -1, +1, +2, +3

Centuries:
1999: 20 Century
2013 : 21 Century
2013: xxx Millenium?

Clock TIme:
23:58 -> 23:59 -> 24:00 -> 0:01
24:00 – 0:1 = 23:59
24:00 + 0:1 = 0:01

For Computations
Very bad idea!

Example, How much is
28. Feb 2000; 23:58:56.123
+ 0:3:12.998
Or
+ 512.998 Secunds?

or time difference from
28. Feb 2000; 23:58:56.123
to
2. April 2000; 12:00:00.000

Time measurement

10^9 ns -> sek
60 sek -> min
60 min -> std
24 std -> tag
28,29,30,31 xTag -> monat
12 Monate -> jahr
365, 366 x Tag -> jahr

Jahreszählung:
-2, -1, +1, +2, +3

Jahrhundert:
1999: 20 Jahrhundert
2013 : 21 Jahrhudert
2013: xxx Jahrtausand?

Uhrzeit:
23:58 -> 23:59 -> 24:00 -> 0:01

For Computations
Very bad idea!

Example, How much is
28. Feb 2000; 23:58:56.123
+ 0:3:12.998
Or
+ 512.998 Secunds?

or time difference from
28. Feb 2000; 23:58:56.123
to
2. April 2000; 12:00:00.000

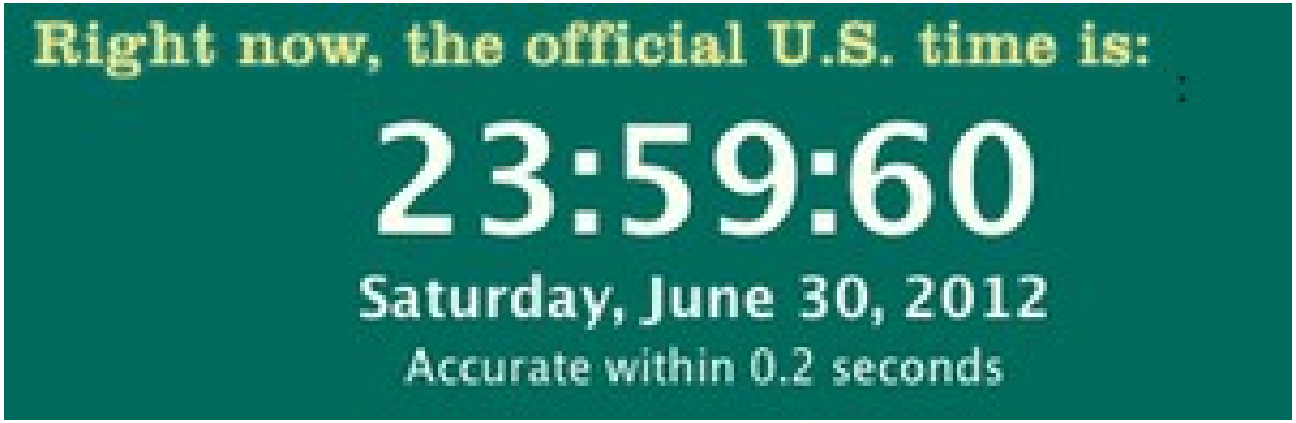
-> We count:
Seconts (+ subseconds)
from:

1. Bootime
2. CycleTime
3. 1.1.2000 um 0:00 (UTC)

Time in control centres
Year, Day-Counter, UTC

Maya calender.....
Time for GPS-Sallits:
Week counter + ns

Leap seconds



Um Sonnensynchron zu bleiben

Leap seconds

UTC = Coordinated Universal Time

UTC = TAI (GPS-Time) + Leap Seconds
TAI = Temps Atomique International
TAI „Constant“ speed, no jumps

0 for TAI = 1.1.1958, (1972: UTC+10)

Today :
UTC = TAI - 35 = GPS - 27 Sek

Greenwich Time = UTC + (0, 1) Hour

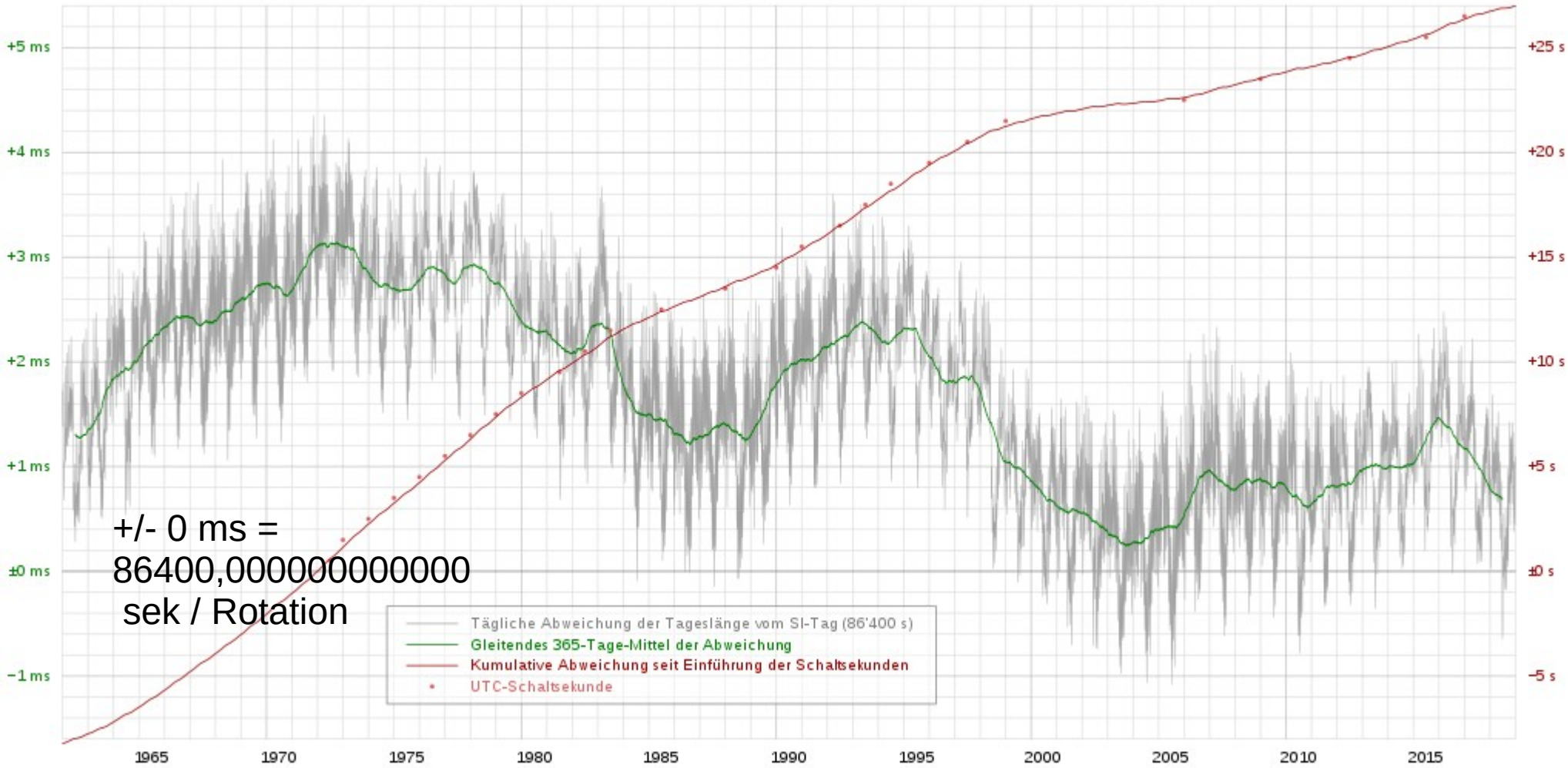
UTC jumps 1 second for each new leap second

UT1 = Sun syncrhnous at Greenwich medirian (0)

Liste aller Schaltsekunden nach 23:59:59 UTC

1972: TAI = UTC + 10 s		
Jahr	**30. Juni**	**31. Dez.**
1972	+ 1 s	+ 1 s
1973	–	+ 1 s
1974	–	+ 1 s
1975	–	+ 1 s
1976	–	+ 1 s
1977	–	+ 1 s
1978	–	+ 1 s
1979	–	+ 1 s
1980	–	–
1981	+ 1 s	–
1982	+ 1 s	–
1983	+ 1 s	–
1984	–	–
1985	+ 1 s	–
1986	–	–
1987	–	+ 1 s
1988	–	–
1989	–	+ 1 s
1990	–	+ 1 s
1991	–	–
1992	+ 1 s	–
1993	+ 1 s	–
1994	+ 1 s	–
1995	–	+ 1 s
1996	–	–
1997	+ 1 s	–
1998	–	+ 1 s
1999	–	–
2000	–	–
1988	–	–
1989	–	+ 1 s
1990	–	+ 1 s
1991	–	–
1992	+ 1 s	–
1993	+ 1 s	–
1994	+ 1 s	–
1995	–	+ 1 s
1996	–	–
1997	+ 1 s	–
1998	–	+ 1 s
1999	–	–
2000	–	–
2001	–	–
2002	–	–
2003	–	–
2004	–	–
2005	–	+ 1 s[3]
2006	–	–
2007	–	–
2008	–	+ 1 s[4]
2009	–	–
2010	–	–
2011	–	–
2012	+ 1 s[5]	–
2013	–	–
2014	–	–
2015	+ 1 s[6]	–
2016	–	+ 1 s[7]
2017	–	–
2018	–	
Σ	**+ 11 s**	**+ 16 s**
	+ 27 s	

Leap Seconds



+/- 0 ms =
86400,000000000000
sek / Rotation

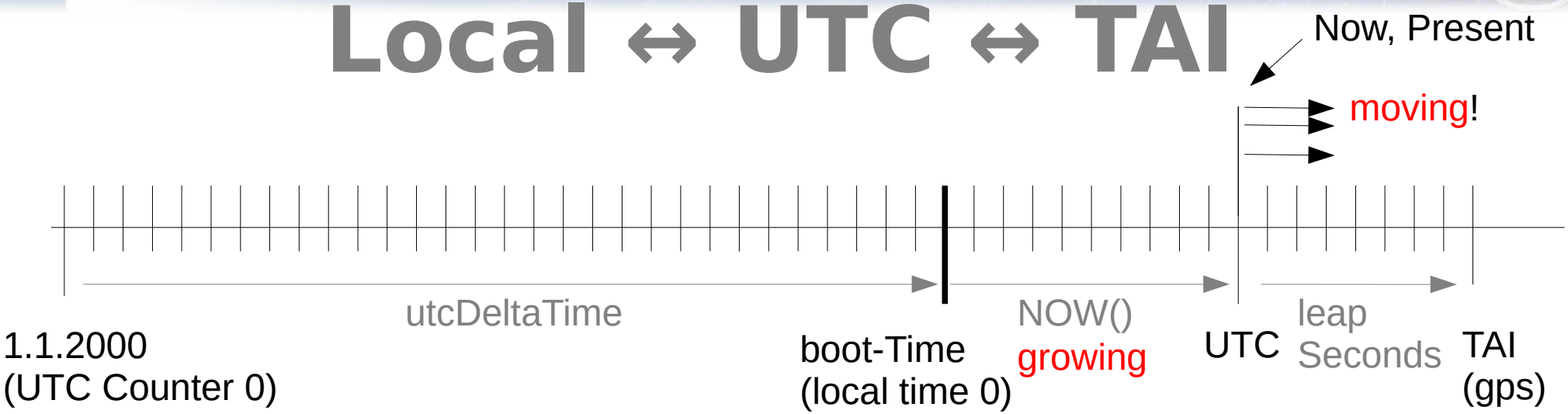
https://de.wikipedia.org/wiki/Schaltsekunde

Example on RODOS

```
// Example of usage: waitFor(3*SECONDS);
// Example           int64_t t0 = 10*HOURS;


#define NANOSECONDS    1LL
#define MICROSECONDS   (1000LL * NANOSECONDS)
#define MILLISECONDS   (1000LL * MICROSECONDS)
#define SECONDS        (1000LL * MILLISECONDS)
#define MINUTES        (60LL * SECONDS)
#define HOURS          (60LL * MINUTES)
#define DAYS           (24LL * HOURS)
#define WEEKS          (7LL  * DAYS)
```

Local ↔ UTC ↔ TAI



utcDeltaTime = Today aprox 400 Millionen Sekunden
NOW() = localTime = 0 .. some Millionen Seconds
LeapSeconds (2019) = aprox. 27

Time measurement

Shall we use
    double seconds
Or better
    int64_t nanoseconds
    int64_t milliseconds

Time measurement

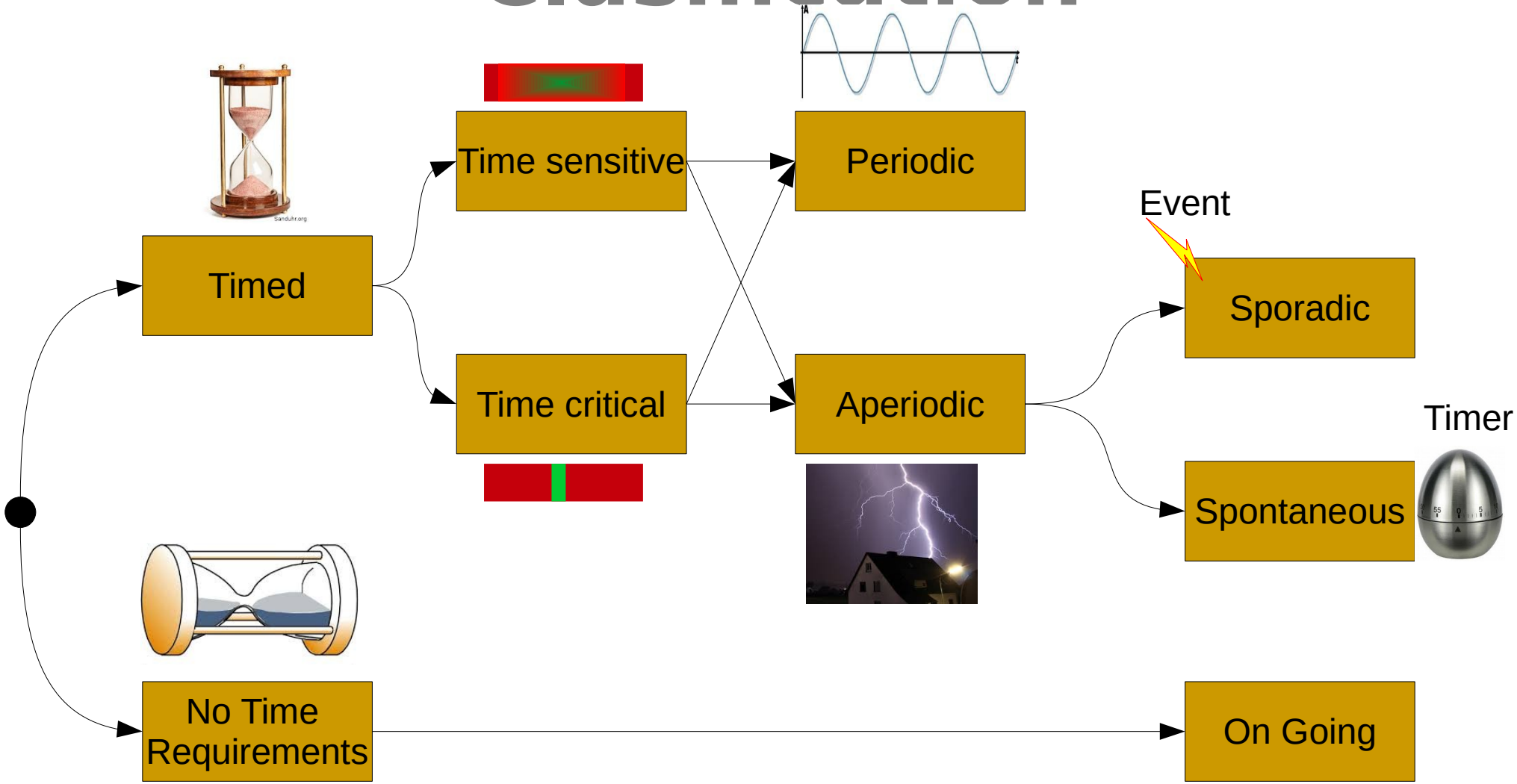
Shall we use
double seconds
Or better
int64_t nanoseconds
int64_t milliseconds

Patriot

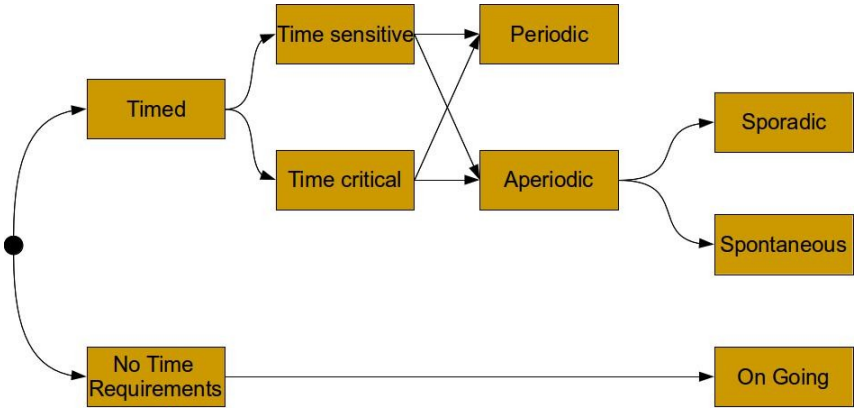
```
float time = 0;
while(1) {
   ....
   time = time + 0.1;
};
```

after 8 hours 300 meter poistion error at mach 3
-> 50 soldiers dead

Real time tasks: Clasification



Real time tasks: Clasification



Normally a control system consists of several task
some
- periodic
- aperiodic
- on going

some
- time critical
- time sensitive
- without time requirements

-> use
- priorities
- dedicated hardware
- Time Space Portioning

mixed systems: Time chritial, time sensitive and without time requirements

In the same computer we may have all 3 types

Time chritial (Hard real time)	highest priority
Time sensitive (Soft real time)	lower priority
Without time requirements (on going)	lowest priority

The time sensitive and the on going task may not interfer with time chritical taskls

→ Timing analyse
→ Priorities

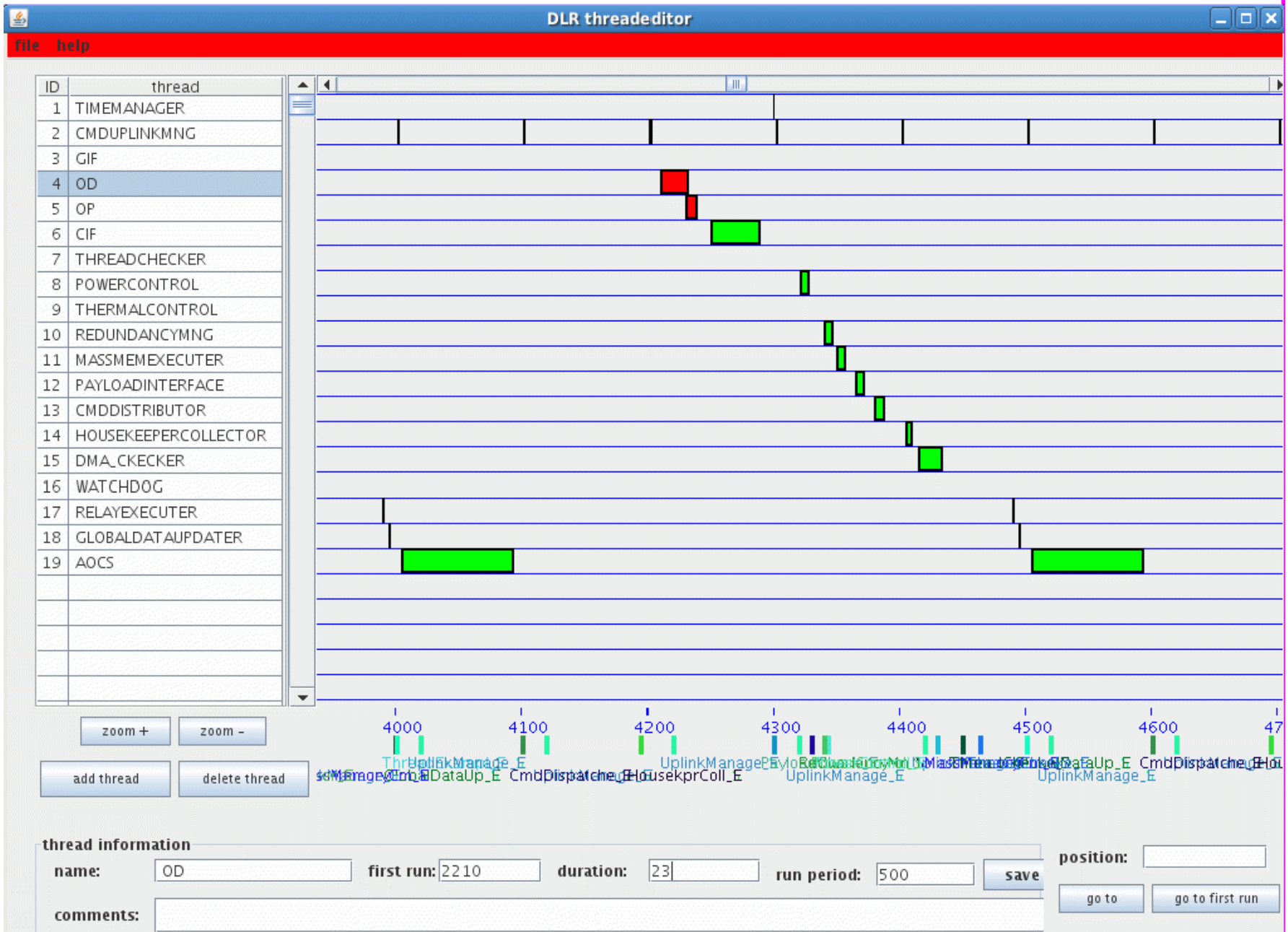
Often:

very time chritical task have own computer/hardware

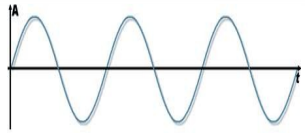
in comming:

Time-Space-Partioning (virtual hardware)

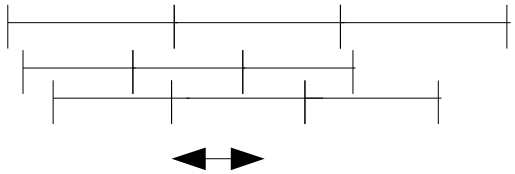
Timing-plan


DLR threadeditor
file
help
ID
thread
1 TIMEMANAGER
2 CMDUPLINKMNG
3 GIF
4 OD
5 OP
6 CIF
7 THREADCHECKER
8 POWERCONTROL
9 THERMALCONTROL
10 REDUNDANCYMNG
11 MASSMEMEXECUTER
12 PAYLOADINTERFACE
13 CMDDISTRIBUTOR
14 HOUSEKEEPERCOLLECTOR
15 DMA_CKECKER
16 WATCHDOG
17 RELAYEXECUTER
18 GLOBALDATAUPDATER
19 AOCS
zoom +
zoom -
add thread
delete thread
4000
4100
4200
4300
4400
4500
4600
thread information
name:
OD
first run:
2210
duration:
23
run period:
500
save
position:
go to
go to first run
comments:

Periodic processes



Periodic processes are activated once per period and need to be finished before the end of the period.



Different tasks may have different period

Period may vary dynamically according to mission phase

Steering / control is executed mostly by periodic tasks

Normally periodic tasks starts executing after boot and continue until the system stops
however is is possible to start and conclude dynamically such tasks (not recommended)

Periodic tasks may perform different functions in each run (eg. state machines)

Aperiodic processes: Sproadic

Handel irregular (sporadic) external events (as interrupts for the CPU)

Mostly they have to react as soon (and fast) as possible.
- they have deadline (latest time point) but no earliest time point
- The Reaction time window begin with the event and ends at the deadline
  - (relative to event entry time point)
    - (eg receive uart data, react to alarms...)

We can not know when the next event will arrive, we can just estimate the highest possible frequency of events
- -> this time has to be free in the scheduler

Interrupts handler / event service routines have a very high priority and have to be very fast (very short execution time)

Aperiodic processes: Spontan

mostly spontan tasks will be activated by other tasks, to control/monitor time out eg:

30 minutes after turning the satellite transmuter on, it has to be off. The device manager would then create an spontaneous task with 30 minutes delay each time it turn the transmitter on.

An aeroplane: The deployment of the landing gear takes about 10 seconds. after issue the command to deploy, 15 seconds later we have to check if the landing gear is out and lock. This would be done by an spontan task after deployment.

In some cases we can use spontan tasks to handle exceptions.

no time requirements: on-going

On Going taks have no time requirements
They have the lowest priority.
They run only if no timed tasks needs CPU time (background)

Eg.

- look for updates ....
- optimise execution parameter,
- memory checks
- check FPGA configuration
- self tests
- thread chekcs
- stack measurements

They have no handle function, they start working and the operating system will supend them when timed task need CPU time

-> **Preemtion**